

PROMPT MÈRE — PHASE 2 : PLATFORM SKELETON

CS GREFFE OS — Strategic Greffe Governance Command Center

Living Governance Engineering

CONTEXTE

Tu es Codex.

Tu travailles sur :

CS GREFFE OS — Strategic Greffe Governance Command Center

dans le cadre méthodologique :

Living Governance Engineering.

Les phases suivantes sont considérées comme terminées :

PHASE 0
Foundation
✓

PHASE 1
Living Memory
✓

La mission actuelle est :

PHASE 2 — PLATFORM SKELETON

PRINCIPE DIRECTEUR

Avant les fonctionnalités vient la structure.

Avant l'intelligence vient l'organisation.

Avant les modules vient le contenant.

La structure constitue le squelette évolutif du système.

OBJECTIF

Construire l'architecture cible de la plateforme.

Préparer les espaces qui accueilleront ultérieurement :

- la connaissance ;
- la gouvernance ;
- Snowflake ;
- le RAG ;
- les agents ;
- les dashboards ;
- les Command Centers.

Sans activer aucune de ces couches.

PRINCIPE FONDAMENTAL

Structure Before Function.

Le squelette doit précéder les capacités.

DOMAINE

Le squelette concerne l'ensemble du dépôt.

RÉPERTOIRES À STRUCTURER

Documentation

```
docs/cs_greffe_os/
```

```
00_FOUNDATIONS/
```

```
01_PROJECT_MEMORY/
```

02_META_GOVERNANCE/
03_ENTERPRISE_ARCHITECTURE/
04_BUSINESS_ARCHITECTURE/
05_GREFFE_OPERATIONS/
06_INFJ/
07_DATA_GOVERNANCE/
08_KNOWLEDGE_GOVERNANCE/
09_AI_GOVERNANCE/
10_SNOWFLAKE/
11_RAG/
12_LLMOPS/
13_MULTI_AGENTS/
14_BUSINESS_MODULES/
15_KPI/
16_COMMAND_CENTER/
17_SECURITY/
18_COMPLIANCE/
19_RESEARCH/
20_ROADMAP/
21_INDUSTRIALIZATION/
22_JDGO/

Source

src/
core/

```
config/  
governance/  
memory/  
catalog/  
metadata/  
observability/  
utilities/
```

Modules

```
modules/  
cs_learn/  
cs_procedure/  
cs_acts/  
cs_registers/  
cs_archives/  
cs_dashboard/  
cs_ethics/  
cs_data/  
cs_admin/  
cs_research/
```

Agents

```
agents/  
supervisor_agent/
```

```
learning_agent/  
procedure_agent/  
draft_agent/  
register_agent/  
archive_agent/  
ethics_agent/  
dashboard_agent/  
data_agent/  
research_agent/
```

Registry

```
registry/  
agent_registry/  
prompt_registry/  
tool_registry/  
workflow_registry/  
model_registry/
```

Prompts

```
prompts/  
system/  
agents/  
modules/  
rag/
```

evaluations/

RAG

rag/
chunking/
embeddings/
retrieval/
reranking/
vectorstores/
knowledge_graph/
evaluation/

LLMOps

llmops/
traces/
logs/
monitoring/
metrics/
evaluations/
experiments/

Streamlit

streamlit_apps/
cs_learn_app/

```
cs_dashboard_app/  
command_center_app/  
executive_dashboard_app/
```

API

```
api/  
fastapi/  
routers/  
schemas/  
services/  
dependencies/
```

Snowflake

```
snowflake/  
roles/  
schemas/  
warehouses/  
tasks/  
streams/  
procedures/  
stages/  
semantic_models/  
cortex/
```

SQL

```
sql/  
  
raw/  
  
curated/  
  
semantic/  
  
ai/  
  
migrations/
```

Data

```
data/  
  
raw/  
  
curated/  
  
semantic/  
  
ai/  
  
snapshots/
```

Dashboards

```
dashboards/  
  
operational/  
  
tactical/  
  
strategic/  
  
executive/
```

Tests

```
tests/  
  
unit/  
  
integration/  
  
regression/  
  
documentation/  
  
rag/  
  
agents/  
  
snowflake/
```

Recherche

```
notebooks/  
  
exploration/  
  
experiments/  
  
research/
```

Livres

```
books/  
  
living_governance_engineering/  
  
cs_greffe_os_handbook/  
  
strategic_greffe_governance_command_center/  
  
judicial_governance_command_center/  
  
digital_justice_research/
```

DIRECTIVES

Créer uniquement :

- des répertoires ;
 - des fichiers `.gitkeep` ;
 - des README ;
 - des index ;
 - des placeholders documentaires.
-

INTERDICTIONS

Ne pas :

- développer Python ;
 - connecter Snowflake ;
 - créer des APIs ;
 - activer Streamlit ;
 - construire le RAG ;
 - créer les agents ;
 - développer les dashboards ;
 - utiliser des données judiciaires réelles.
-

OUVERTURE FUTURE

Le squelette doit pouvoir accueillir :

Français

↓

Français + Anglais

↓

Multilingue

et supporter :

- Snowflake ;
- Multi-Agent Systems ;
- RAG ;
- LLMOps ;
- nouveaux modèles ;
- futures technologies ;

- futurs modules.
-

PORTABILITÉ

Le squelette doit rester :

- simple ;
 - modulaire ;
 - versionnable ;
 - portable ;
 - extensible ;
 - indépendant des outils.
-

STYLE

Architecture durable.

Pas d'optimisation prématurée.

Pas de sophistication inutile.

Toujours privilégier :

Structure Before Function.

CRITÈRES DE SORTIE

La phase est terminée lorsque :

- ✓ l'arborescence cible existe ;
 - ✓ les domaines documentaires sont créés ;
 - ✓ les modules sont préparés ;
 - ✓ les espaces futurs sont réservés ;
 - ✓ aucune fonctionnalité prématurée n'a été activée ;
 - ✓ l'architecture reste extensible ;
 - ✓ le système est prêt à recevoir la connaissance.
-

COMMITTS

Convention :

```
PHASE_<n>_<HHMM>: <description>
```

Exemples :

```
PHASE_2_0815: initialize platform skeleton  
  
PHASE_2_0840: scaffold platform architecture  
  
PHASE_2_1015: create documentation domains  
  
PHASE_2_1140: complete platform skeleton
```

PRINCIPE DE CONTINUITÉ

Conformément au Principe Fondamental n°8 :

Every completed phase must prepare the next one.

La clôture de la PHASE 2 devra préparer :

PHASE 3 — KNOWLEDGE FOUNDATION

car :

```
Foundation  
↓  
Memory  
↓  
Platform  
↓  
Knowledge  
↓  
Governance  
↓  
Data  
↓  
AI  
↓  
Agents
```

↓
Applications
↓
Dashboards
↓
Command Centers

MAXIME DE LA PHASE 2

Le squelette ne produit pas encore d'intelligence.

Mais il rend possible son émergence future.

Toujours privilégier :

Memory Before Code.

Governance Before Automation.

Structure Before Function.

Humans Before AI.

Identity Before Complexity.

Traceability Before Performance.